
Un Problema de Cuadratura



Integrantes del Grupo:
Martin Cerda Fernandez
Maximiliano Campos Camimil

Docente: Sebastián Salazar Molina
Fecha de entrega: 15 de junio de 2026

Introducción.....	4
1. Descripción de la Solución implementada.....	5
1.1 Arquitectura General.....	5
1. Descarga SFTP (dos fases).....	5
2. Parseo de CSVs.....	5
3. Consulta a la API REST con caché persistente.....	6
4. Acumulación con reduction.....	6
5. Cálculo y salida.....	6
1.2 Flujo o Pseudocódigo del algoritmo paralelo.....	7
2. Estructura del Proyecto.....	9
2.1 Organización de Directorios.....	9
2.2 Módulos del Programa.....	10
3. Implementación Técnica.....	10
3.1 Conexión SFTP.....	11
3.2 Estructura de los Archivos CSV.....	12
Estrategia de lectura y validación.....	13
Manejo de filas inválidas o corruptas.....	14
Eficiencia del parseo.....	14
3.3 API REST.....	14
3.4 Justificación de Librerías Externas.....	17
4. Implementación con OpenMP.....	18
4.1 Puntos de Paralelización.....	18
4.2 Pseudocódigo Paralelo.....	20
Descarga SFTP.....	20
Parseo de CSVs + Consulta API + Acumulación.....	21
Cálculo de métricas y salida.....	23
4.3 Salida Esperada del Programa.....	24
Conexión y escaneo SFTP:.....	24
Inicialización de la API:.....	25
Procesamiento paralelo de CSVs:.....	25
Resultados finales:.....	25
5. Manejo de Errores y Robustez.....	27
5.1 Errores Contemplados.....	27
5.2 Formato del Archivo log.txt.....	28
6. Sistema de Construcción (Makefile).....	29
6.1 Targets del Makefile (all, clean, run).....	29
6.2 Flags de compilación utilizados.....	30
7. Resultados y Análisis.....	32

7.1 Salida del Programa.....	32
7.2 Análisis de Rendimiento.....	33
8. Dificultades Encontradas.....	35
8.1 Limitación de thread-safety en libssh2.....	35
8.2 Rechazo de conexiones simultáneas por el servidor (MaxStartups).....	35
8.3 Volumen de consultas a la API y necesidad de caché en dos niveles.....	35
8.4 Tamaño de la caché y límite de archivos en GitHub.....	35
8.5 Filas malformadas en los archivos CSV.....	35
8.6 Sincronización del token JWT en entorno paralelo.....	36
9. Conclusión.....	37

Introducción

En el ámbito del procesamiento de grandes volúmenes de datos, la computación paralela se presenta como una herramienta fundamental para reducir los tiempos de ejecución y aprovechar al máximo los recursos del hardware moderno. El presente informe documenta el diseño e implementación de una solución en C++ que aplica paralelismo mediante la biblioteca OpenMP para resolver un problema de cuadratura orientado al análisis de datos de ventas.

El sistema desarrollado integra múltiples tecnologías: acceso a un servidor remoto mediante el protocolo SFTP para la descarga de archivos de reporte en formato CSV, consultas a una API REST externa para obtener información demográfica de los clientes, y procesamiento concurrente de los datos para calcular el promedio de montos de compra segmentado por género.

El objetivo central es demostrar cómo la paralelización de tareas independientes como descarga, parseo, consulta y acumulación permite obtener resultados de forma eficiente, midiendo el impacto real del paralelismo a través de métricas de tiempo de ejecución y speedup.

A lo largo del informe se describe la arquitectura de la solución, la estructura del proyecto, las especificaciones técnicas de cada componente, las convenciones de código aplicadas y finalmente los resultados obtenidos junto a su análisis.

1. Descripción de la Solución implementada

1.1 Arquitectura General

La solución se estructura como un pipeline de cinco etapas implementado en C++, donde el paralelismo es introducido mediante OpenMP en aquellas etapas donde las tareas son independientes entre sí. El diseño sigue el principio de separación de responsabilidades: cada módulo (sftp, csv, api) encapsula su propia lógica y expone una interfaz mínima hacia “main.cpp”, que actúa como orquestador general.

1. Descarga SFTP (dos fases)

La descarga no es simplemente un **parallel for** sobre todos los archivos, sino que se divide en dos fases deliberadamente distintas:

- Fase 1 (secuencial): Se establece una única sesión SSH para recorrer el directorio raíz del servidor y construir la lista de archivos “reporte_*.csv” que aún no existen en el directorio local “csv_files/”. Los archivos ya descargados son saltados, lo que convierte cada ejecución en incremental.
- Fase 2 (paralela): Solo los archivos faltantes son descargados. Se lanza un “#pragma omp parallel for schedule(dynamic,1) num_threads(8)” donde cada hilo establece su propia conexión TCP + sesión SSH independiente. El límite de 8 hilos fue una decisión de diseño explícita para no saturar el parámetro “MaxStartups” del servidor OpenSSH, que rechaza conexiones simultáneas excesivas. La variable “descargas_nuevas” se acumula con “reduction(+:descargas_nuevas)”.

2. Parseo de CSVs

Una vez descargados, “main.cpp” lista los archivos locales con “std::filesystem::directory_iterator” y lanza el loop principal paralelo:

C/C++

```
#pragma omp parallel for schedule(dynamic, 1)
    reduction(+: suma_f, suma_m, count_f, count_m)
```

Cada hilo invoca “csv::parsear(archivo)”, que lee el archivo completo de forma local y retorna un “std::vector<Transaccion>”. El parseo usa un separador, (no coma), maneja campos entre comillas y valida que los headers coincidan exactamente con los 10 esperados. Las filas malformadas o con errores de conversión son descartadas y registradas en **log.txt**. Esta etapa **no requiere secciones críticas** porque cada hilo trabaja sobre su propio vector local antes de acumular.

3. Consulta a la API REST con caché persistente

Por cada transacción, el hilo llama a “api::consultar(uuid)”, que implementa la siguiente estrategia:

1. **Caché en memoria:** Un “std::unordered_map<string, Genero>” protegido con un “omp_lock_t”. Si el UUID ya fue consultado, se retorna directamente sin tráfico de red.
2. **Caché en disco:** Al iniciar el programa, “api::init()” carga el archivo “cache_generos.txt” (si existe), precargando resultados de ejecuciones anteriores. Al terminar, “api::cleanup()” serializa la caché de vuelta al disco.
3. **Token JWT con renovación automática:** Antes de cada petición, se verifica si el token expira en menos de 60 segundos, de ser así, se renueva de forma segura con omp_lock_t. Si se recibe un HTTP 401 durante una consulta, se fuerza renovación inmediata.
4. **Handles thread_local:** Cada hilo mantiene su propia instancia “thread_local CURL*” con “TCP_KEEPALIVE” activo, evitando el overhead de crear y destruir un handle por cada petición.
5. **Política de reintentos:** Ante fallos de red o HTTP 5xx, se realizan hasta 3 intentos con 500ms de espera entre ellos antes de descartar el UUID y registrar el error.

4. Acumulación con reduction

Dentro del mismo loop paralelo de la Etapa 2, cada hilo acumula en variables locales (lsf, lsm, lcf, lcm) los montos y conteos por género, y al final del bloque los suma a las variables compartidas. La directiva “reduction(+: suma_f, suma_m, count_f, count_m)” de OpenMP garantiza que esta combinación se haga sin condiciones de carrera y sin necesidad de “omp critical” explícito sobre los acumuladores.

5. Cálculo y salida

Al salir del bloque paralelo, “main.cpp” calcula los promedios, imprime el resultado en consola y escribe “resultados.txt”. El tiempo total se mide con “omp_get_wtime()” desde el inicio hasta este punto, incluyendo descarga, parseo y consultas.

Cabe notar que si todos los archivos “reporte_*.csv” ya están presentes en “csv_files/” (ejecución no inicial), la Fase 1 de descarga se completa casi instantáneamente (no hay Fase 2), por lo que “TIEMPO” refleja principalmente el costo de parseo, consultas a la API (mitigadas por la caché) y cálculo de métricas.

1.2 Flujo o Pseudocódigo del algoritmo paralelo

A continuación, el pseudocódigo del núcleo paralelo del programa, la región que integra parseo de CSV, consulta a la API y acumulación de resultados:

INICIO

t_inicio <- omp_get_wtime()

Descarga SFTP

Fase 1 (secuencial, 1 sesión SSH):

conectar al servidor SFTP

listar archivos remotos "reporte_*.csv"

faltantes <- archivos remotos que NO existen en csv_files/

Fase 2 (paralela, 8 hilos fijos):

PARA CADA archivo EN faltantes EN PARALELO (8 hilos, dynamic, 1):

abrir conexión TCP + sesión SSH propia del hilo

descargar archivo a csv_files/

descargas_nuevas += 1 // reduction(+:descargas_nuevas)

Inicialización API

api::init()

cargar cache_generos.txt en RAM (unordered_map UUID -> Genero)

hacer_login() -> obtener token JWT

archivos <- listar csv_files/*.csv (ordenados)

suma_f, suma_m, count_f, count_m <- 0

Región paralela principal

PARA CADA archivo[i] EN PARALELO (schedule dynamic, 1)

reduction(+: suma_f, suma_m, count_f, count_m):

transacciones <- csv::parsear(archivo[i])

Si transacciones vacío: CONTINUAR

lsf, lsm, lcf, lcm <- 0 // acumuladores LOCALES del hilo

PARA CADA t EN transacciones:

Consulta de genero (con cache)

Si t.codigo_cliente está en cache (lock cache):

g <- cache[t.codigo_cliente]

SINO:

asegurar_token() // renueva si expira en <60s

g <- GET /person/{uuid} // hasta 3 reintentos

Si éxito: cache[t.codigo_cliente] <- g (lock cache)

Acumulación local según genero

Si g == FEMENINO: lsf += t.monto_aplicado; lcf += 1

Si g == MASCULINO: lsm += t.monto_aplicado; lcm += 1

g == DESCONOCIDO -> transacción ignorada

combinar acumuladores locales con los globales via reduction

```
suma_f += lsf; suma_m += lsm  
count_f += lcf; count_m += lcm
```

Resultados

```
prom_f <- suma_f / count_f  
prom_m <- suma_m / count_m  
tiempo <- omp_get_wtime() - t_inicio
```

imprimir/guardar FEMENINO, MASCULINO, TIEMPO en consola y resultados.txt

api::cleanup()

guardar cache_generos.txt actualizado en disco

2. Estructura del Proyecto

2.1 Organización de Directorios

La estructura del repositorio separa el código fuente de los archivos generados en tiempo de compilación y ejecución. Los directorios “obj/” y “csv_files/”, junto con el binario “cruz_morada” y “cache_generos.txt”, están excluidos del control de versiones mediante “.gitignore” (ya que son artefactos pesados o regenerables localmente). Se incluyen en el repositorio “cache_generos.zip” (caché pre-poblada comprimida), “resultados.txt” y “log.txt” de una ejecución de referencia, como evidencia del comportamiento real del sistema.

cruz-morada-parallel/

	├── include/ ← Cabeceras de los módulos
	├── api.h ← Enum Genero, namespace api (init/consultar/cleanup)
	├── csv.h ← Struct Transaccion, namespace csv::parsear()
	└── sftp.h ← namespace sftp::descargar_archivos()
	├── src/ ← Implementaciones
	├── main.cpp ← Orquestador: pipeline, OpenMP, salida
	├── api.cpp ← JWT, caché, consultas HTTP con libcurl
	├── csv.cpp ← Parseo y validación de archivos CSV
	└── sftp.cpp ← Descarga SFTP en dos fases con libssh2
	├── obj/ ← [generado por make] Objetos compilados
	├── csv_files/ ← [generado al ejecutar] CSVs descargados del SFTP
	├── cruz_morada ← [generado por make] Binario ejecutable
	├── cache_generos.txt ← [generado al descomprimir/ejecutar] Caché UUID→género
	├── cache_generos.zip ← Caché pre-poblada comprimida (incluida en el repo)
	├── resultados.txt ← Salida de referencia (incluida en el repo)
	├── log.txt ← Log de referencia (incluido en el repo)
	├── Makefile ← Sistema de construcción (targets: all, clean)
	├── README.md ← Instrucciones de compilación y uso
	└── .gitignore ← Excluye binarios, objetos y archivos regenerables

2.2 Módulos del Programa

El programa se compone de cuatro módulos con responsabilidades claramente delimitadas, más el orquestador principal. Cada módulo expone únicamente lo necesario en su cabecera `.h`, manteniendo el resto como implementación interna.

Módulo	Función Principal	Responsabilidad
sftp	descargar_archivos()	Conexión SFTP y descarga de CSVs
csv	parsear()	Lectura y parseo de archivos CSV
api	consultar(), init(), cleanup()	Consultas REST al endpoint /person/{uuid}
main	main()	Orquestación general, OpenMP, salida de resultados

El programa se compone de cuatro módulos con responsabilidades claramente delimitadas, más el orquestador principal. Cada módulo expone únicamente lo necesario en su cabecera `“.h,”` manteniendo el resto como implementación interna.

Tipos de datos compartidos entre módulos definidos en las cabeceras para que todos los módulos los conozcan sin depender unos de otros:

Tipo	Definido en	Descripción
struct Transaccion	include/csv.h	Representa una fila del CSV con sus 10 campos tipados (fecha, canal, sku, producto, unidades, porcentaje_descuento, monto_aplicado, boleta, local, codigo_cliente)
enum class Genero	include/api.h	Tres valores posibles: FEMENINO, MASCULINO, DESCONOCIDO. Los registros con DESCONOCIDO son ignorados en la acumulación

3. Implementación Técnica

3.1 Conexión SFTP

Para la conexión con el servidor remoto se utilizó libssh2 directamente, sin intermediación de libcurl. Esta decisión se tomó porque libcurl, si bien soporta SFTP internamente a través de libssh2, agrega una capa de abstracción que dificulta el control explícito del ciclo de vida de cada sesión SSH por hilo — necesario para que, en la Fase 2, cada hilo abra y cierre su propia conexión independiente sin compartir estado con los demás. Al usar libssh2 directamente se tiene control total sobre el ciclo de vida de cada conexión, lo que es indispensable para la estrategia de descarga paralela implementada.

Estrategia de descarga

La función “sftp::descargar_archivos()” divide el trabajo en dos fases secuenciales entre sí, pero con naturaleza distinta:

Fase 1 — Escaneo secuencial

Se establece una única conexión al servidor para listar el contenido del directorio raíz. El flujo de esta fase es:

```
C/C++
libssh2_init(0)
├─ conectar_tcp()           ← socket TCP al host 137.184.45.251:22
├─ libssh2_session_init()
│   └─ libssh2_session_handshake()
│       └─ libssh2_userauth_password()
│           └─ libssh2_sftp_init()
│               └─ libssh2_sftp_opendir("/")
│                   └─ libssh2_sftp_readdir() [loop]
```

Por cada entrada del directorio, el programa verifica tres condiciones antes de agregarlo a la lista de faltantes:

- Que el nombre comience con reporte_ (strncmp)
- Que la extensión sea .csv (strcmp sobre los últimos 4 caracteres)
- Que el archivo no exista ya en csv_files/ (intento de fopen local)

Los archivos que ya existen en disco son contados como “descargados_previos” y saltados. Esto hace que cada ejecución sea incremental: solo se transfieren los archivos nuevos del servidor, sin repetir descargas ya realizadas.

Al terminar el escaneo, **toda la conexión de esta fase es cerrada explícitamente** antes de iniciar la fase paralela, evitando que hilos compartan una sesión SSH que no es thread-safe.

Fase 2 — Descarga paralela

Con la lista de archivos faltantes construida, se lanza el siguiente bloque paralelo:

```
C/C++
#pragma omp parallel for schedule(dynamic, 1) num_threads(8) \
    reduction(+: descargas_nuevas)
for (int i = 0; i < total_faltantes; i++) {
    // Cada hilo crea su propia conexión TCP + sesión SSH + sesión SFTP
    int t_sock = conectar_tcp();
    LIBSSH2_SESSION* t_session = libssh2_session_init();
    libssh2_session_handshake(t_session, t_sock);
    libssh2_userauth_password(t_session, SFTP_USER, SFTP_PASS);
    LIBSSH2_SFTP* t_sftp = libssh2_sftp_init(t_session);

    bajar_archivo(t_sftp, remoto, local);

    // Cada hilo cierra su propia conexión al terminar
    libssh2_sftp_shutdown(t_sftp);
    libssh2_session_disconnect(t_session, "Normal shutdown");
    libssh2_session_free(t_session);
    close(t_sock);
}
```

3.2 Estructura de los Archivos CSV

Cada archivo descargado desde el servidor representa el registro de transacciones de un día específico. El nombre sigue la nomenclatura “reporte_(Año)(DíaDelAño).csv”, donde el día se expresa como número ordinal del año. Por ejemplo, “reporte_26128.csv” corresponde al día 128 del año 2026, equivalente al 8 de mayo de 2026.

Columna	Tipo en CSV	Tipo en C++	Descripción
FECHA	String ISO 8601	std::string	Fecha y hora de la transacción (UTC-4)
CANAL	String	std::string	Canal de compra (ej: POS, online)
SKU	Integer	int	Identificador único del producto
PRODUCTO	String	std::string	Nombre del producto
UNIDADES	Integer	int	Cantidad de unidades compradas

PORCENTAJE DESCUENTO	Float (0 a 1)	double	Descuento aplicado sobre el precio
MONTO APLICADO	Float	double	Monto total pagado en CLP (campo clave)
BOLETA	Integer	long	Número de boleta de la transacción
LOCAL	String	std::string	Identificador del local de compra
CODIGO CLIENTE	UUID v4	std::string	Identificador único del cliente (campo clave)

De las 10 columnas, el programa utiliza únicamente dos para el cálculo de métricas: MONTO APLICADO como valor a acumular, y CODIGO CLIENTE como clave para consultar el género del cliente en la API. El resto de los campos son parseados y almacenados en el struct Transaccion pero no intervienen en el resultado final.

Estrategia de lectura y validación

Validación de cabecera: Al abrir el archivo, se lee la primera línea y se compara campo a campo contra el vector “HEADERS_ESPERADOS”, que contiene los 10 nombres exactos en el orden exacto esperado. Si el número de columnas no coincide, o cualquier nombre difiere, el archivo completo es descartado y el evento queda registrado en “log.txt”. Esto protege contra archivos con estructura incorrecta o versiones distintas del formato.

C/C++

```
static const std::vector<std::string> HEADERS_ESPERADOS = {
    "FECHA", "CANAL", "SKU", "PRODUCTO", "UNIDADES",
    "PORCENTAJE DESCUENTO", "MONTO APLICADO", "BOLETA", "LOCAL", "CODIGO CLIENTE"
};
```

Validación estructural por fila: Cada línea es dividida por “split_linea()”, que maneja correctamente campos encerrados entre comillas dobles (donde el separador ; puede aparecer dentro del valor sin romper el parseo). Si una fila no produce exactamente 10 campos, es descartada individualmente sin interrumpir el procesamiento del resto del archivo.

Validación de tipos: Los campos numéricos (SKU, UNIDADES, PORCENTAJE DESCUENTO, MONTO APLICADO, BOLETA) son convertidos con “std::stoi”, “std::stod” y “std::stol” dentro de un bloque try/catch. Cualquier fallo de conversión (valor vacío, texto en campo numérico, desbordamiento) descarta la fila y registra el error con el número de fila exacto en “log.txt”.

Adicionalmente, la función elimina caracteres \r al final de cada línea antes de procesarla, garantizando compatibilidad con archivos generados en sistemas Windows que usan \r\n como fin de línea.

Manejo de filas inválidas o corruptas

El programa nunca aborta por una fila malformada.

Situación	Acción	Registro en log.txt
Archivo no existe o no se puede abrir	Se descarta el archivo completo	[CSV] ruta: No se pudo abrir
Archivo vacío (sin cabecera)	Se descarta el archivo completo	[CSV] ruta: Archivo vacío
Número de columnas en cabecera incorrecto	Se descarta el archivo completo	[CSV] ruta: Numero de columnas incorrecto: N
Nombre de columna no coincide	Se descarta el archivo completo	[CSV] ruta: Header incorrecto: esperado 'X' encontrado 'Y'
Fila con número de campos distinto de 10	Se descarta solo esa fila	[CSV] ruta: Fila N malformada (M campos)
Error de conversión de tipo en un campo	Se descarta solo esa fila	[CSV] ruta: Fila N error conversion: mensaje

Eficiencia del parseo

El vector de resultado tiene una llamada a “resultado.reserve(30000)” antes del loop de lectura. Este valor fue determinado empíricamente como una cota superior razonable para el número de transacciones diarias de la farmacia, evitando realocaciones repetidas del vector durante el parseo de archivos grandes. Dado que cada hilo trabaja sobre su propio vector local (retornado por valor desde “csv::parsear”), esta operación no requiere ningún tipo de sincronización.

3.3 API REST

Propósito y endpoint utilizado

Los CSV solo tienen el UUID del cliente, no dicen si es hombre o mujer. Para eso existe el endpoint “GET /v1/person/{uuid}” que devuelve un JSON con los datos de la persona. De todo lo que retorna (nombre, rut, fecha de nacimiento, etc.), lo único que nos importa es el campo “gender”, que puede valer “FEMENINO” o “MASCULINO”.

URL base: <https://api.sebastian.cl/cpyd>

Endpoint: GET /v1/person/{uuid}

Auth: Bearer JWT (header Authorization)

Respuesta: { "gender": "FEMENINO" | "MASCULINO" }

Autenticación JWT

La API no es pública, hay que autenticarse primero. Se hace un POST a “/v1/login/authenticate” con el email y rut del integrante, y el servidor devuelve un token JWT. Ese token se manda en cada petición en el header “Authorization: Bearer <token>”.

El problema es que el token expira, entonces hay que estar pendiente de renovarlo. Lo que hicimos fue leer el campo “exp” del payload del JWT (que viene en base64) y compararlo con la hora actual antes de cada consulta. Si faltan menos de 60 segundos para que expire, se renueva antes de hacer la petición. Esto lo implementamos nosotros mismos sin librería externa, decodificando el base64 a mano, porque el payload tiene una estructura simple y no valía la pena agregar una dependencia solo para eso.

También puede pasar que el token expire justo en medio de una consulta y el servidor devuelva 401. En ese caso el programa detecta el 401, renueva el token y reintenta automáticamente.

Como todo esto se ejecuta en paralelo con OpenMP, protegimos el token con un “omp_lock_t” para que si varios hilos detectan que expiró al mismo tiempo, solo uno haga el login y los demás esperen:

```
C/C++  
omp_set_lock(&g_token_lock);  
  
if (g_token.empty() || (g_token_exp - ahora) < 60) {  
    hacer_login();  
}  
  
omp_unset_lock(&g_token_lock);
```

La caché

Sin caché el programa sería inviable. Un mismo cliente puede aparecer cientos de veces en distintos archivos CSV, y si consultáramos la API por cada fila el tiempo de ejecución sería enorme (y además le pegaríamos innecesariamente al servidor).

La solución fue un “unordered_map<string, Genero>” en memoria que guarda los UUID ya consultados. Elegimos “unordered_map” sobre “map” porque la búsqueda es O(1) promedio en vez

de $O(\log n)$, y como estamos hablando de potencialmente miles de UUIDs distintos esa diferencia importa.

El flujo es simple: antes de hacer cualquier petición HTTP, se revisa si el UUID ya está en el mapa. Si está, se devuelve directo. Si no está, se consulta la API y se guarda el resultado para la próxima vez.

Como varios hilos pueden estar leyendo y escribiendo el mapa al mismo tiempo, lo protegimos con otro “omp_lock_t”. Lo importante es que el lock se toma solo durante el acceso al mapa, no durante la petición HTTP, así los hilos no se bloquean entre sí mientras esperan respuesta del servidor:

consultar(uuid)

├— revisar caché (con lock) → si está, retornar directo

├— hacer petición HTTP (sin lock, en paralelo)

└— guardar resultado en caché (con lock)

Además de la caché en RAM, implementamos persistencia en disco mediante “cache_generos.txt”. Al iniciar el programa se carga ese archivo si existe, y al terminar se guarda todo lo que se acumuló. Esto significa que en la segunda ejecución en adelante, prácticamente no hay peticiones a la API porque todos los UUIDs ya están en disco. El formato es simple, un UUID por línea seguido de F o M:

e74ce7d1-ea0d-37ca-a209-1d86522e7df5 F

fddd14ef-3a67-31f3-93cd-64a05fd41b41 M

Reintentos y manejo de errores de red

La latencia de la API es variable y a veces simplemente falla o tarda demasiado. Para no perder transacciones por un error puntual, implementamos hasta 3 intentos por UUID con 500ms de espera entre ellos. Si los 3 fallan, el UUID queda como DESCONOCIDO y esa transacción se ignora en el cálculo final, lo que registramos en “log.txt”.

Respuesta	Qué hace el programa
HTTP 200	Extrae gender y guarda en caché
HTTP 401	Renueva el token y reintent
Otro error / timeout	Espera 500ms y reintent (hasta 3 veces)

3 fallos seguidos	Registra en log y descarta la transacción
-------------------	---

Handles de curl por hilo y parsing JSON

Para las peticiones usamos libcurl. En vez de crear un handle nuevo por cada consulta, usamos “thread_local CURL*”, o sea cada hilo tiene su propio handle que reutiliza en todas sus peticiones. Así no hay que hacer el handshake TLS cada vez y el tiempo de cada consulta baja bastante.

Para parsear el JSON tampoco usamos ninguna librería externa. Hicimos una función pequeña “json_get()” que busca un campo por nombre y extrae su valor. Como la respuesta siempre tiene la misma estructura y solo necesitamos un campo, una función de 15 líneas fue suficiente y no tiene sentido añadir una dependencia completa como “nlohmann/json” para algo tan puntual.

3.4 Justificación de Librerías Externas

libssh2

Es la librería que permite hablar el protocolo SSH/SFTP desde C++. La usamos para conectarnos al servidor, listar el directorio raíz y descargar los archivos CSV.

La alternativa más obvia era usar libcurl con soporte SFTP (que internamente igual usa libssh2), pero el problema es que libcurl abstrae demasiado: no permite manejar sesiones SSH de forma individual por hilo, que era exactamente lo que necesitábamos para que cada hilo de la descarga paralela tuviera su propia conexión independiente. Con libssh2 directo tenemos control total sobre el ciclo de vida de cada sesión.

libcurl

La usamos exclusivamente para las peticiones HTTP a la API REST. Es la librería más estándar que existe para esto en C/C++, tiene soporte para HTTPS, manejo de headers, timeouts y keepalive, que son cosas que necesitábamos.

Consideramos hacer las peticiones a mano con sockets y TLS, pero eso habría sido reinventar la rueda innecesariamente y el código habría quedado mucho más complejo y propenso a errores.

4. Implementación con OpenMP

4.1 Puntos de Paralelización

El programa tiene dos zonas donde se usa paralelismo real con OpenMP, más varias estructuras de sincronización que protegen los recursos compartidos entre hilos. Aquí explicamos cada una:

Descarga de archivos (sftp.cpp)

Una vez que el escaneo secuencial construye la lista de archivos faltantes, se lanza un “parallel for” donde cada hilo descarga un archivo distinto. Como cada hilo crea su propia conexión SSH independiente, no hay nada compartido en la transferencia en sí.

C/C++

```
#pragma omp parallel for schedule(dynamic, 1) num_threads(8) \
    reduction(+: descargas_nuevas)
```

El límite de 8 hilos es fijo y deliberado: más conexiones simultáneas al servidor SSH empezaban a ser rechazadas.

Procesamiento de CSVs (main.cpp)

Es el loop principal del programa. Cada hilo toma un archivo CSV, lo parsea completo y por cada transacción consulta el género del cliente en la API. Todo ocurre dentro del mismo bloque paralelo.

C/C++

```
#pragma omp parallel for schedule(dynamic, 1) \
    reduction(+: suma_f, suma_m, count_f, count_m)
```

Se usa “schedule(dynamic, 1)” porque los archivos tienen tamaños distintos. Con planificación estática algunos hilos terminarían mucho antes que otros y quedarían ociosos.

Tabla de directivas y sincronización

Etapas	Directiva OpenMP	Qué se sincroniza	Mecanismo
Descarga SFTP	#pragma omp parallel for num_threads(8) schedule(dynamic,1)	Contador de descargas exitosas	reduction(+: descargas_nuevas)
Descarga SFTP (log)	#pragma omp critical	Escritura en log.txt desde hilos	Sección crítica en log_err()

Parseo CSV	Dentro del parallel for de main	Nada, cada hilo trabaja en su propio vector<Transaccion> local	No requiere sincronización
Consulta API — caché	Dentro del parallel for de main	Lectura y escritura del unordered_map compartido	omp_lock_t g_cache_lock
Consulta API — token JWT	Dentro del parallel for de main	Lectura, escritura y renovación del token global	omp_lock_t g_token_lock
Acumulación de montos	Dentro del parallel for de main	suma_f, suma_m, count_f, count_m	reduction(+: ...)

Por qué “omp_lock_t” y no “omp critical” para la caché y el token

Al principio la caché la protegíamos con “#pragma omp critical”, que es más simple de escribir. El problema es que “critical” bloquea a todos los hilos aunque estén accediendo a secciones críticas distintas (si no se les pone nombre). Con “omp_lock_t” el lock existe solo sobre la estructura específica que se está protegiendo, y lo más importante: se puede soltar el lock justo después de consultar el mapa y antes de hacer la petición HTTP. Así los hilos no se bloquean entre sí mientras esperan respuesta del servidor, que es exactamente el momento donde más tiempo se pierde.

Por qué “reduction” y no “atomic” para los acumuladores

Teníamos la opción de usar “#pragma omp atomic” sobre cada suma, pero “reduction” es más eficiente en este caso porque cada hilo acumula en su propia copia local de las variables durante todo el loop y solo las combina al final. Con “atomic” habría una operación atómica por cada transacción procesada, lo que genera mucha contención cuando hay muchos hilos sumando al mismo tiempo.

4.2 Pseudocódigo Paralelo

El pseudocódigo está estructurado en cada etapa del programa, destacando las directivas OpenMP utilizadas y la lógica de sincronización aplicada. El objetivo es mostrar, de forma clara y abstracta, cómo se organiza el paralelismo en cada fase sin entrar en los detalles sintácticos del lenguaje C++. Cada etapa fue diseñada de manera independiente para facilitar su comprensión y mantenimiento.

Descarga SFTP

La descarga de archivos se divide en dos fases. Primero, se establece una conexión secuencial al servidor para listar todos los archivos disponibles y filtrar aquellos que aún no existen en el directorio local "csv_files/". Esta fase es inherentemente secuencial dado que el escaneo del directorio remoto no puede paralelizarse sin múltiples conexiones activas. Una vez obtenida la lista de archivos faltantes, se procede a descargarlos en paralelo: cada hilo establece su propia conexión SSH/SFTP independiente, descarga el archivo asignado y lo escribe en disco. El número de hilos se limita a 8 para evitar saturar el parámetro "MaxStartups" del servidor remoto.

C/C++

```
INICIO sftp::descargar_archivos()
```

```
    //Escaneo secuencial (un solo hilo)
```

```
    sock      ← conectar_tcp(SFTP_HOST, SFTP_PORT)
```

```
    session   ← libssh2_session_init()
```

```
    libssh2_session_handshake(session, sock)
```

```
    libssh2_userauth_password(session, SFTP_USER, SFTP_PASS)
```

```
    sftp_sess ← libssh2_sftp_init(session)
```

```
    dir       ← libssh2_sftp_opendir(sftp_sess, "/")
```

```
    faltantes ← []
```

```
    PARA CADA entrada EN dir:
```

```
        SI nombre comienza con "reporte_" Y termina en ".csv":
```

```
            SI archivo NO existe en csv_files/:
```

```
                agregar nombre a faltantes
```

```
    cerrar dir, sftp_sess, session, sock  // conexión de escaneo cerrada
```

```
    //Descarga paralela (solo archivos faltantes)
```

```
    #pragma omp parallel for schedule(dynamic, 1) num_threads(8)
```

```
        reduction(+: descargas_nuevas)
```

PARA CADA archivo EN faltantes:

```
t_sock    ← conectar_tcp()           // conexión propia por hilo
t_session ← libssh2_session_init()
libssh2_session_handshake(t_session, t_sock)
libssh2_userauth_password(t_session, SFTP_USER, SFTP_PASS)
t_sftp    ← libssh2_sftp_init(t_session)
```

SI bajar_archivo(t_sftp, remoto, local) == OK:

```
descargas_nuevas++
```

```
cerrar t_sftp, t_session, t_sock      // cada hilo cierra su
conexión
```

FIN sftp::descargar_archivos()

Parseo de CSVs + Consulta API + Acumulación

Esta es la etapa central del programa y la que más se beneficia del paralelismo. Cada hilo recibe un archivo CSV distinto, lo parsea completamente y, por cada transacción válida encontrada, consulta el género del cliente a través de la API REST. Para evitar consultas redundantes, se utiliza una caché compartida protegida con un lock de OpenMP, de modo que un UUID ya resuelto no genera una nueva petición HTTP. Los acumuladores de suma y conteo por género se manejan mediante la cláusula “reduction”, lo que elimina la necesidad de secciones críticas para las variables numéricas y reduce el overhead de sincronización.

C/C++

INICIO procesamiento principal (main.cpp)

```
archivos ← listar_csvs("csv_files/")
api::init(RUT, EMAIL)           // carga caché desde disco +
login JWT
```

```
suma_f = 0, suma_m = 0
count_f = 0, count_m = 0
```

```
#pragma omp parallel for schedule(dynamic, 1)
```

```

        reduction(+: suma_f, suma_m, count_f, count_m)
PARA CADA archivo EN archivos:

// -- Parseo CSV (local por hilo, sin sincronización) --
transacciones ← csv::parsear(archivo)
  abrir archivo
  validar headers (exactamente 10 columnas)
  SI headers inválidos: registrar en log.txt, descartar archivo
  PARA CADA línea:
    campos ← split_linea(línea, separador=';')
    SI campos != 10: registrar en log.txt, descartar fila
    INTENTAR convertir tipos numéricos
    SI error conversión: registrar en log.txt, descartar fila
    agregar Transaccion al vector local
  retornar vector<Transaccion>

// -- Consulta API con caché (por cada transacción) --
lsf = 0, lsm = 0, lcf = 0, lcm = 0
PARA CADA t EN transacciones:

  genero ← api::consultar(t.codigo_cliente)
  omp_set_lock(g_cache_lock)
  SI uuid EN caché:
    omp_unset_lock(g_cache_lock)
    retornar caché[uuid] // sin tráfico de red
  omp_unset_lock(g_cache_lock)

  asegurar_token() // renueva si expira en < 60s
  PARA intento EN [1..3]:
    respuesta ← HTTP GET /v1/person/{uuid}
    SI HTTP 200:
      genero ← extraer "gender" del JSON
      omp_set_lock(g_cache_lock)

```

```

        caché[uuid] ← genero
        omp_unset_lock(g_cache_lock)
        BREAK
    SI HTTP 401:
        renovar token, reintentar
    SI otro error:
        esperar 500ms, reintentar
    SI 3 fallos: registrar en log.txt, retornar DESCONOCIDO

    // -- Acumulación local por hilo --
    SI genero == FEMENINO: lsf += t.monto_aplicado; lcf++
    SI genero == MASCULINO: lsm += t.monto_aplicado; lcm++

    // reduction combina al salir del bloque paralelo
    suma_f += lsf; suma_m += lsm
    count_f += lcf; count_m += lcm

FIN bloque paralelo    // OpenMP combina las copias de cada hilo aquí

```

Cálculo de métricas y salida

Una vez finalizado el bloque paralelo, todos los acumuladores parciales han sido combinados automáticamente por OpenMP a través de la reducción. En esta etapa, que es completamente secuencial, se calculan los promedios dividiendo la suma total de montos por el conteo de transacciones para cada género. Finalmente, se imprime el resultado por consola y se escribe el archivo “resultados.txt”. El tiempo total de ejecución se obtiene restando el timestamp inicial, capturado con “omp_get_wtime()” al comienzo del programa, del timestamp final registrado en este punto.

```

C/C++

// Ejecutado por un solo hilo tras el bloque paralelo
prom_f ← suma_f / count_f
prom_m ← suma_m / count_m
tiempo ← omp_get_wtime() - t_inicio

```

```
IMPRIMIR "FEMENINO  = {prom_f}"
IMPRIMIR "MASCULINO = {prom_m}"
IMPRIMIR "TIEMPO    = {tiempo} segundos"

ESCRIBIR resultados.txt

api::cleanup()    // serializa caché a disco (cache_generos.txt)
```

4.3 Salida Esperada del Programa

Durante la ejecución, el programa emite mensajes informativos por consola que permiten seguir el progreso de cada etapa en tiempo real. Esta salida está diseñada para facilitar la depuración y verificar que el pipeline se ejecuta correctamente, desde la conexión al servidor remoto hasta el cálculo final de métricas. A continuación se describe lo que el usuario puede esperar ver en cada fase.

Conexión y escaneo SFTP:

Al iniciarse, el programa establece una conexión secuencial al servidor SFTP e informa sobre el estado del escaneo remoto. Se notifica cuántos archivos ya existen en disco y cuántos son nuevos y deben descargarse. Si todos los archivos ya están presentes localmente, el programa lo indica y omite la fase de descarga, pasando directamente al procesamiento. En caso de iniciarse la descarga paralela, cada hilo reporta en consola el nombre del archivo que acaba de obtener.

```
Shell
[INFO] Conectando SFTP...
[INFO] Escaneando servidor remoto...
[SFTP] Se detectaron 910 archivos nuevos.
[SFTP] Iniciando descarga paralela (Limite: 8 hilos)...
[Hilo 0] Descargó: reporte_26001.csv
[Hilo 3] Descargó: reporte_26002.csv
[Hilo 1] Descargó: reporte_26003.csv
...
[SFTP] Total: 910 archivos disponibles
```

En ejecuciones posteriores, donde los archivos ya existen localmente, esta fase se reduce a:

```
Shell
[INFO] Conectando SFTP...
[INFO] Escaneando servidor remoto...
[SFTP] Entorno al dia. No hay archivos nuevos.
```



```
[SFTP] Total archivos en disco: 910
```

Inicialización de la API:

Una vez completada la descarga, el programa inicializa el módulo de consumo de la API REST. Esto incluye la carga de la caché local desde disco, si existe el archivo “cache_generos.txt” de una ejecución previa, y la obtención del token JWT mediante una petición de login. La consola muestra cuántos registros fueron recuperados desde la caché y cuántos segundos restan antes de que el token expire, lo que permite verificar que la autenticación fue exitosa antes de iniciar el procesamiento.

Shell

```
[API] Cache local cargada: 2924736 registros.  
[API] Token obtenido, expira en 1020 segundos
```

Si no existe “cache_generos.txt” la primera línea no aparece y se realizan consultas HTTP para cada UUID único encontrado.

Procesamiento paralelo de CSVs:

Durante la etapa central de procesamiento, cada hilo reporta el archivo CSV que está procesando junto con la cantidad de transacciones válidas que encontró en él. Esta salida es concurrente, por lo que el orden de los mensajes puede variar entre ejecuciones dependiendo del scheduling de OpenMP. Esta información es útil para identificar archivos con volúmenes de datos inusualmente altos o bajos, y para confirmar que todos los archivos están siendo procesados correctamente.

Shell

```
[INFO] 910 archivos CSV  
[INFO] Hilos: 8  
[Hilo 0] csv_files/reporte_26001.csv -> 16923 tx  
[Hilo 2] csv_files/reporte_26002.csv -> 17105 tx  
[Hilo 5] csv_files/reporte_26003.csv -> 16788 tx  
...
```

Resultados finales:

Al concluir el bloque paralelo, el programa imprime un bloque de resultados claramente delimitado con el promedio de compras calculado para cada género y el tiempo total de ejecución medido con “omp_get_wtime()”. Adicionalmente, se muestra una línea de depuración con el conteo total de transacciones procesadas por género, lo que permite validar la consistencia de los resultados. Estos mismos valores quedan registrados en el archivo “resultados.txt” para su entrega.

Shell

```
=== RESULTADOS ===  
FEMENINO = 12488
```

```
MASCULINO = 13076
TIEMPO    = 30.7577 segundos
=====
[DEBUG] TX Femenino: XXXXX | Masculino: XXXXX
[OK] resultados.txt guardado
[API] Cache local guardada en disco.
```

Contenido del archivo **resultados.txt**:

```
Shell
FEMENINO = 12488
MASCULINO = 13076
TIEMPO = 30.7577 segundos
```

Contenido del archivo **log.txt**:

```
Shell
[CSV] csv_files/reporte_24136.csv: Fila 5777 malformada (9 campos)
[CSV] csv_files/reporte_24300.csv: Fila 1805 malformada (4 campos)
[CSV] csv_files/reporte_24312.csv: Fila 29935 malformada (2 campos)
```

Si no se llegan a presentar errores de red ni archivos corruptos adicionales, **log.txt** contiene únicamente las filas malformadas detectadas durante el parseo. Un **log.txt** vacío indica una ejecución sin anomalías.

5. Manejo de Errores y Robustez

El sistema fue diseñado bajo la premisa de que ningún error individual (un archivo corrupto, una consulta API fallida, un token expirado) debe detener el procesamiento completo de los ~910 archivos y ~15.4 millones de transacciones. Cada módulo implementa su propia función “log_err()”, que escribe en un archivo “log.txt” compartido con un prefijo identificando el origen del error ([SFTP], [CSV], [API], [MAIN]), y además imprime el mismo mensaje por stderr para visibilidad inmediata durante la ejecución.

Dado que múltiples hilos pueden escribir errores simultáneamente, la escritura a “log.txt” está protegida: en el módulo SFTP mediante “#pragma omp critical”, y en los módulos CSV y API mediante apertura/cierre atómico del archivo en modo “a” (append) por cada mensaje, evitando que las líneas de distintos hilos se entremezclan de forma corrupta.

5.1 Errores Contemplados

Errores de conexión SFTP:

- Falla en la resolución del host o conexión TCP al servidor (No se pudo resolver host / No se pudo conectar al servidor SFTP).
- Falla en el handshake SSH o en la autenticación usuario/contraseña.
- Falla al abrir el directorio remoto o al abrir un archivo remoto específico para lectura.
- Falla al crear el archivo local de destino (permisos, espacio en disco).
- En la Fase 2 (descarga paralela), si un hilo individual no logra conectarse o autenticarse, ese hilo omite su archivo (continue) sin afectar a los demás hilos ni abortar la descarga completa.

Errores de parseo CSV:

- Archivo inexistente o no legible (No se pudo abrir: <motivo>).
- Archivo vacío (sin encabezados).
- Encabezados que no coinciden en cantidad o nombre con las 10 columnas esperadas — en este caso el archivo completo se descarta y se registra el error, pero el procesamiento de los demás archivos continúa.
- Filas individuales con un número de campos distinto a 10 (por ejemplo, reporte_24136.csv fila 5777 con 9 campos, o reporte_24312.csv fila 29935 con 2 campos, ambos detectados durante las pruebas reales). Estas filas se omiten individualmente sin descartar el resto del archivo.
- Errores de conversión de tipos (std::stoi, std::stod, std::stol lanzando excepción) en columnas numéricas (SKU, UNIDADES, PORCENTAJE DESCUENTO, MONTO APLICADO, BOLETA) — la fila se omite y se registra el error con el motivo de la excepción.

Errores de la API REST:

- Fallo de login (`hacer_login()`) retorna false si la petición HTTP falla o si la respuesta no contiene un campo `jwt`.
- Token expirado durante la ejecución (HTTP 401): el código detecta este caso, fuerza la renovación del token (`g_token_exp = 0`) y reintenta la consulta automáticamente. En la ejecución de referencia el token obtenido tuvo una vida útil de ~1020 segundos (ver consola: "Token obtenido, expira en 1020 segundos"), superior al tiempo total de procesamiento (~30 segundos), por lo que no se produjeron renovaciones en esta corrida particular. Dado que el token tiene una vida útil de ~1020 segundos (~17 minutos), en escenarios donde el procesamiento requiere mucho más tiempo (por ejemplo, una ejecución con caché de géneros vacía, consultando ~2.9 millones de clientes únicos), el mecanismo de renovación automática (`g_token_exp - ahora < 60`) se activaría decenas de veces de forma transparente, sin intervención manual y sin interrumpir el procesamiento.
- Timeouts o errores de red en `curl_easy_perform()` (`CURLE_*` distinto de OK): se reintenta hasta 3 veces con una espera de 500ms entre intentos antes de marcar el cliente como género DESCONOCIDO.
- Códigos HTTP distintos de 200/401 (por ejemplo 404 para UUIDs no encontrados): se registra el código recibido y el cliente se trata como DESCONOCIDO.
- Respuestas 200 cuyo campo `gender` no es "FEMENINO" ni "MASCULINO": se registra el valor recibido.

Tratamiento de clientes con género desconocido: siguiendo la especificación del enunciado "ignorar transacciones con UUIDs no encontrados en la API o con género no definido", las transacciones cuyo cliente resultó en género DESCONOCIDO no se incluyen en los acumuladores "suma_f", "suma_m", "count_f", "count_m", pero no generan un error de ejecución: el programa continúa normalmente.

5.2 Formato del Archivo log.txt

Cada línea de "log.txt" sigue el formato "[MODULO] contexto: mensaje", donde "MODULO" identifica el origen (SFTP, CSV, API, MAIN). Ejemplos reales obtenidos durante la ejecución de referencia:

```
[CSV] csv_files/reporte_24300.csv: Fila 1805 malformada (4 campos)
[CSV] csv_files/reporte_24312.csv: Fila 29935 malformada (2 campos)
[CSV] csv_files/reporte_24136.csv: Fila 5777 malformada (9 campos)
```

El archivo "log.txt" se abre en modo append ("a") y se reinicia únicamente al ejecutar "make clean", lo que permite que registre errores acumulados de múltiples ejecuciones si así se desea, o partir de un archivo limpio tras "make clean && make". Se incluye en el repositorio un "log.txt" de referencia correspondiente a la ejecución completa sobre los ~910 archivos, como evidencia del comportamiento real del sistema de logging ante un dataset de producción.

6. Sistema de Construcción (Makefile)

6.1 Targets del Makefile (all, clean, run)

El Makefile implementa dos targets principales: “all” y “clean”. Vale mencionar que no se implementó un target “run” explícito, ya que el binario requiere que las dependencias del sistema (libssh2 y libcurl) estén instaladas y que el servidor SFTP sea accesible, así que ejecutarlo directamente con make no aportaba nada útil sobre simplemente llamar al binario. Un target “run” que falla silenciosamente por dependencias faltantes o por inaccesibilidad del servidor SFTP podría ser más confuso que útil, y el README cubre cómo ejecutar el binario directamente.

Target all: Es el target por defecto. Hace dos cosas en orden: primero crea el directorio “obj/” si no existe, y luego compila cada “.cpp” de “src/” como objeto independiente antes de linkearlos en el binario final “cruz_morada”.

makefile

```
None
all: $(OBJ_DIR) $(BIN)

$(OBJ_DIR):
    mkdir -p $(OBJ_DIR)

$(BIN): $(OBJS)
    $(CXX) $(OBJS) -o $@ $(LDFLAGS)
    @echo ">>> Compilado: $(BIN)"

$(OBJ_DIR)/%.o: $(SRC_DIR)/%.cpp
    $(CXX) $(CXXFLAGS) -I$(INC_DIR) -c $< -o $@
```

La regla de compilación usa “\$(wildcard \$(SRC_DIR)/*.cpp)” para detectar automáticamente todos los archivos fuente en “src/”, y “\$(patsubst ...)” para mapear cada “.cpp” a su “.o” correspondiente en “obj/”. Esto significa que si se agrega un nuevo módulo al proyecto, el Makefile lo compila solo sin necesidad de modificarlo.

Se usa compilación separada por objeto (no todo en un solo comando) para aprovechar la recompilación incremental: si solo se modifica “api.cpp”, make recompila únicamente “api.o” y re-linkea, sin tocar “csv.o” ni “sftp.o”.

Para compilar:

bash

```
Shell
make
# o equivalentemente
make all
```

Target clean: Elimina todo lo generado: el directorio “obj/” con los objetos compilados, el binario “cruz_morada”, y los archivos de salida “resultados.txt” y “log.txt”.

makefile

None

```
clean:
    rm -rf $(OBJ_DIR) $(BIN) resultados.txt log.txt
    @echo ">>> Limpio"
```

bash

Shell

make clean

Notar que “clean” no elimina “csv_files/” ni “cache_generos.txt”. Esto es intencional, los CSV descargados y la caché de géneros son datos que tomaron tiempo obtener (descarga SFTP y consultas a la API respectivamente), y borrarlos junto con los objetos compilados sería innecesariamente costoso. Si se quiere hacer una limpieza total incluyendo esos archivos, hay que hacerlo manualmente.

6.2 Flags de compilación utilizados

Los flags están definidos en dos variables del Makefile: “CXXFLAGS” para la compilación de cada objeto, y “LDFLAGS” para el linkeo final.

makefile

None

```
CXX          = g++
CXXFLAGS     = -Wall -Wextra -O2 -fopenmp -std=c++17
LDFLAGS      = -fopenmp -lssh2 -lcurl
```

Flags de compilación (CXXFLAGS)

-std=c++17: Habilita el estándar C++17. Lo necesitamos específicamente por “std::filesystem”, que usamos en “main.cpp” para listar los archivos CSV del directorio local con “fs::directory_iterator”. En estándares anteriores esto no existe como parte de la librería estándar.

-fopenmp: Activa el soporte de OpenMP en el compilador. Sin este flag todos los “#pragma omp” son ignorados silenciosamente y el programa corre en un solo hilo sin advertir nada, lo cual es un error difícil de detectar si no se sabe que hay que activarlo explícitamente.

-O2: Nivel de optimización 2. Le indica al compilador que aplique optimizaciones de rendimiento sin llegar a “-O3”, que puede hacer el debug más difícil porque reorganiza agresivamente el código. Con “-O2” el compilador hace cosas como inlining de funciones pequeñas, eliminación de código

muerto y optimización de loops, lo que ayuda bastante en el parseo de CSV donde hay mucha iteración.

-Wall: Activa los warnings más comunes: variables no usadas, comparaciones entre tipos distintos, funciones sin return, etc. Lo pusimos desde el principio para mantener el código limpio.

-Wextra: Agrega warnings adicionales que “-Wall” no cubre, como parámetros de función no usados o comparaciones entre signed e unsigned. Junto con “-Wall” cubre prácticamente todos los errores comunes que el compilador puede detectar sin ejecutar el programa.

Flags de linkeo (LDFLAGS)

-fopenmp: Aparece tanto en “CXXFLAGS” como en “LDFLAGS” porque OpenMP necesita estar presente en ambas etapas. En compilación activa los pragmas, en linkeo enlaza la librería runtime de OpenMP (“libgomp”) que implementa la gestión real de hilos en tiempo de ejecución.

-lssh2: Enlaza la librería libssh2 para todo lo relacionado con SSH y SFTP: handshake, autenticación por contraseña, apertura de directorios remotos y descarga de archivos.

-lcurl: Enlaza libcurl para las peticiones HTTPS a la API REST. Internamente libcurl se encarga de TLS, manejo de headers y keepalive.

7. Resultados y Análisis

7.1 Salida del Programa

Tras procesar la totalidad de los archivos “reporte_*.csv” (previamente descargados y presentes en “csv_files/”), con la caché de géneros “cache_generos.txt” precargada desde ejecuciones anteriores, el programa produjo los siguientes resultados:

None

```
=== RESULTADOS ===  
FEMENINO  = 12488  
MASCULINO = 13076  
TIEMPO    = 30.7577 segundos  
=====
```

[DEBUG] TX Femenino: 10521806 | Masculino: 4875421

Los valores obtenidos indican que el **promedio de monto de compra de clientes femeninos fue de \$12.488 CLP**, mientras que el **promedio de monto de compra de clientes masculinos fue de \$13.076 CLP**, lo que representa una diferencia de \$588 CLP a favor del segmento masculino.

Estos mismos valores fueron escritos automáticamente en el archivo “resultados.txt” con el siguiente contenido:

None

```
FEMENINO = 12488  
MASCULINO = 13076  
TIEMPO = 30.7577 segundos
```

El tiempo total de ejecución de 30.7577 segundos corresponde a una ejecución donde los archivos CSV ya estaban presentes en “csv_files/” y la caché de géneros “cache_generos.txt” estaba precargada, por lo que este tiempo refleja principalmente el costo del parseo paralelo de los ~910 archivos y el cálculo de métricas con “reduction”. La fase de descarga SFTP fue casi instantánea al no haber archivos nuevos en el servidor, y no se generaron consultas nuevas a la API (ningún cliente fuera de la caché), por lo que “log.txt” no contiene entradas de tipo “[API]” para esta ejecución — únicamente las advertencias de filas CSV malformadas, detalladas en la sección 5.2.

7.2 Análisis de Rendimiento

Modo	Hilos	Tiempo (s)	Speedup	Eficiencia
Secuencia	1	91.16	1.00×	100%
Paralelo	2	63.68	1.43×	71.5%
Paralelo	4	51.81	1.76×	44.0%
Paralelo	8	40.44	2.25×	28.2%

Ganancia real del paralelismo

Al pasar de 1 a 8 hilos, el tiempo de ejecución se redujo de 91.16 s a 40.44 s, logrando un speedup de **2.25×**. Si bien la mejora es apreciable, se encuentra muy por debajo del speedup ideal de 8×, lo que evidencia que el programa no es puramente CPU-bound.

Cuello de botella: las consultas a la API REST

El principal limitante del rendimiento es la latencia de red asociada a las consultas al endpoint `/v1/person/{uuid}`. El programa procesó un total de 15.397.227 transacciones (10.521.806 femeninas + 4.875.421 masculinas), y aunque el caché en disco (`cache_generos.txt`) evitó consultas redundantes entre ejecuciones, las peticiones HTTP introducen un overhead de I/O que no se beneficia del paralelismo de la misma manera que el cómputo puro. Cada hilo espera la respuesta de la red, limitando la utilización efectiva del CPU.

Rendimiento decreciente con más hilos

La eficiencia cae de 71.5% con 2 hilos a 28.2% con 8 hilos. Esto responde a dos fenómenos combinados:

- **Contención en los locks de OpenMP:** el caché compartido (`g_cache`) y el token JWT (`g_token`) utilizan `omp_lock_t`. A mayor número de hilos, los tiempos de espera para adquirir estos locks aumentan.
- **Saturación del servidor:** con 8 conexiones simultáneas al servidor SFTP y a la API REST, se genera mayor contención en el lado del servidor, aumentando los tiempos de respuesta individuales.

Etapas más beneficiada por el paralelismo

La etapa de descarga SFTP fue la que más se benefició directamente de la paralelización, ya que el módulo `sftp.cpp` limita explícitamente las descargas a 8 hilos concurrentes con `num_threads(8)`, reduciendo el tiempo de descarga de forma casi lineal respecto a la cantidad de archivos nuevos. El **parseo CSV**, al ser una operación puramente en disco y CPU, también mejoró de forma consistente con el aumento de hilos.

Conclusión del análisis

El programa cumple el objetivo del paralelismo: reduce el tiempo total de ~91 s a ~40 s, un ahorro de más de 50 segundos. Sin embargo, la Ley de Amdahl establece que la fracción secuencial del programa (principalmente la latencia de red) impone un techo al speedup. Para superar esta limitación, una mejora futura sería implementar solicitudes HTTP asíncronas (con “libcurl multi-handle”) que permitan a cada hilo iniciar múltiples peticiones en paralelo sin bloquear mientras espera la respuesta del servidor.

8. Dificultades Encontradas

8.1 Limitación de thread-safety en libssh2

La primera aproximación para paralelizar la descarga SFTP fue intentar compartir una única sesión SSH entre múltiples hilos OpenMP. Esto no es viable: “libssh2” no es thread-safe sobre una misma conexión, provocando fallos impredecibles en tiempo de ejecución. La solución fue diseñar una arquitectura de dos fases: una conexión secuencial única para escanear el servidor y construir la lista de archivos faltantes (Fase 1), seguida de una región paralela donde cada hilo crea su propia conexión TCP y sesión SSH completamente independiente (Fase 2). Esto eliminó el problema de thread-safety sin sacrificar el paralelismo.

8.2 Rechazo de conexiones simultáneas por el servidor (MaxStartups)

Una vez resuelta la separación de sesiones por hilo, surgió una segunda limitación: el servidor OpenSSH rechazaba conexiones cuando demasiados hilos intentaban autenticarse simultáneamente, debido al parámetro “MaxStartups” del servidor. Se realizaron pruebas con distintos valores y se determinó que “num_threads(8)” era el límite seguro que maximizaba la velocidad de descarga sin generar rechazos. Este valor está fijo en “sftp.cpp” con un comentario explicativo, de modo que cualquier evaluador entienda que no es un número arbitrario sino una decisión basada en el comportamiento real del servidor.

8.3 Volumen de consultas a la API y necesidad de caché en dos niveles

Con ~910 archivos CSV y ~15.4 millones de transacciones, consultar la API por cada fila habría sido inviable. La solución fue implementar una caché de dos niveles: en memoria mediante “unordered_map<string, Genero>” protegido con “omp_lock_t” para evitar consultas duplicadas dentro de la misma ejecución, y persistente en disco mediante “cache_generos.txt” para que ejecuciones posteriores no repitan consultas ya resueltas. El diseño del lock fue especialmente importante: se libera inmediatamente después de consultar el mapa y antes de hacer la petición HTTP, de modo que los hilos no se bloquean entre sí durante la espera de respuesta del servidor.

8.4 Tamaño de la caché y límite de archivos en GitHub

La caché de géneros completamente poblada (“cache_generos.txt”) alcanzó aproximadamente 114 MB, superando el límite de 100 MB por archivo que impone GitHub. La solución fue comprimirla como “cache_generos.zip” (~64 MB) e incluirla en el repositorio junto con instrucciones en el “README.md” para descomprimirla antes de ejecutar el programa. Esto permite que cualquier persona que clone el repositorio tenga acceso inmediato a la caché pre-poblada, evitando que la primera ejecución deba resolver todos los clientes únicos desde cero consultando la API.

8.5 Filas malformadas en los archivos CSV

Durante la ejecución real sobre los ~910 archivos se detectaron filas con número de campos incorrecto en al menos tres archivos: “reporte_24136.csv” fila 5777 con 9 campos, “reporte_24300.csv” fila 1805 con 4 campos, y “reporte_24312.csv” fila 29935 con 2 campos. Esto obligó a implementar validación fila por fila dentro de “csv::parsear()”, descartando individualmente

las filas malformadas sin interrumpir el procesamiento del resto del archivo. Todos estos casos quedan registrados en "log.txt" con el número de fila exacto, lo que permitió verificar que el sistema los manejaba correctamente sin afectar el resultado final.

8.6 Sincronización del token JWT en entorno paralelo

El token JWT tiene una vida útil de ~1020 segundos. En escenarios donde la caché no está poblada y el programa debe consultar millones de clientes únicos, el token puede expirar durante la ejecución. El problema en un contexto paralelo es que múltiples hilos pueden detectar simultáneamente que el token expiró y todos intentar renovarlo al mismo tiempo. La solución fue proteger la renovación con "omp_lock_t g_token_lock:" el primer hilo que detecta la expiración entra al lock, renueva el token y lo libera, los demás hilos que lleguen mientras tanto esperan y al obtener el lock verifican que ya fue renovado, evitando logins redundantes. Adicionalmente, si el servidor devuelve HTTP 401 en medio de una consulta, se fuerza la renovación inmediata y se reintenta automáticamente.

9. Conclusión

El presente trabajo demostró que es posible construir un sistema de procesamiento de datos de alto volumen en C++ que integra múltiples tecnologías —SFTP, REST API y paralelismo con OpenMP— de forma robusta y eficiente. Se procesaron exitosamente ~910 archivos CSV con ~15.4 millones de transacciones, obteniendo como resultado un promedio de compra de **\$12.488 CLP para clientes femeninos y \$13.076 CLP para clientes masculinos** en un tiempo total de **30.7577 segundos** con la caché de géneros precargada.

El impacto real del paralelismo se manifestó en distintas etapas del pipeline. La descarga SFTP en Fase 2 permitió transferir múltiples archivos simultáneamente con 8 hilos concurrentes, reduciendo significativamente el tiempo respecto a una descarga secuencial. El procesamiento de CSVs con “schedule(dynamic, 1)” distribuyó eficientemente la carga entre hilos considerando que los archivos tienen tamaños distintos. La caché de géneros en dos niveles fue probablemente la optimización de mayor impacto: al evitar consultas HTTP redundantes, convirtió una tarea potencialmente dominada por latencia de red en una operación mayormente local.

En cuanto a las decisiones de diseño, el uso de “reduction” sobre variables acumuladoras resultó más adecuado que “atomic” para este caso, y el uso de “omp_lock_t” sobre “#pragma omp critical” permitió liberar lpartidos con mayor granularidad, evitando que los hilos se bloquearan durante las peticiones HTTP, que es el punto donde más tiempo se pierde.

Entre las limitaciones actuales de la solución, la más relevante es que el programa no puede paralelizar la Fase 1 de descarga SFTP (el escaneo del directorio remoto), que por su naturaleza debe ejecutarse de forma secuencial. Adicionalmente, el límite de 8 hilos en la descarga no es configurable en tiempo de ejecución sino que está definido como constante en el código, lo que podría ser un problema en servidores con configuraciones distintas de “MaxStartups”.

Como mejoras futuras se identifican principalmente tres: implementar un target “run” en el Makefile que gestione automáticamente la descompresión de “cache_generos.zip” antes de ejecutar, hacer configurable el número máximo de hilos de descarga mediante un argumento de línea de comandos, y explorar el uso de “std::async” o una pool de hilos más sofisticada para la etapa de consultas a la API, lo que permitiría desacoplar el parseo CSV de las peticiones HTTP y potencialmente solapar ambas operaciones en el tiempo.